ADVANCED SQL QUERY SYSTEM

Purplle Online Beauty Shopping Database Management System

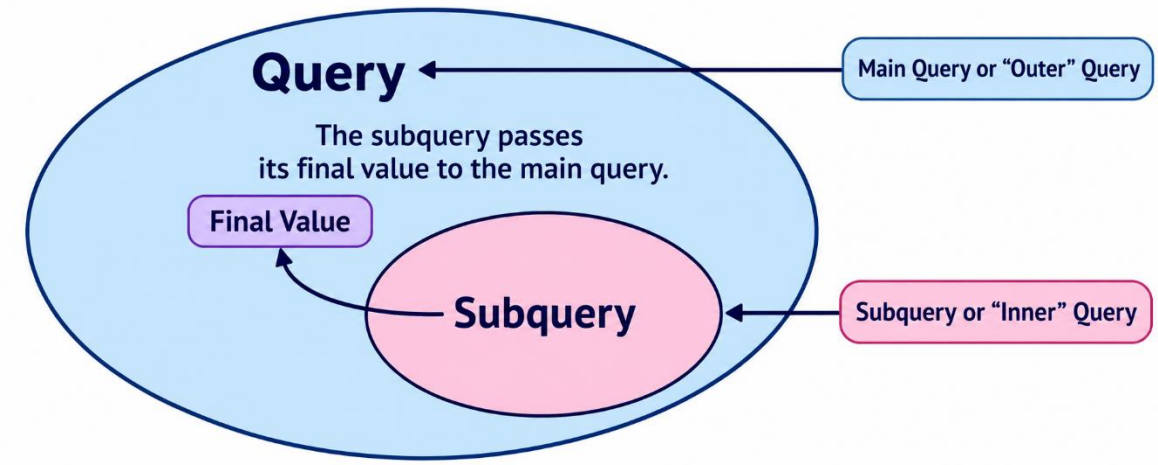
Objectives

1. Implement subqueries and nested queries.
2. Find products above the average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

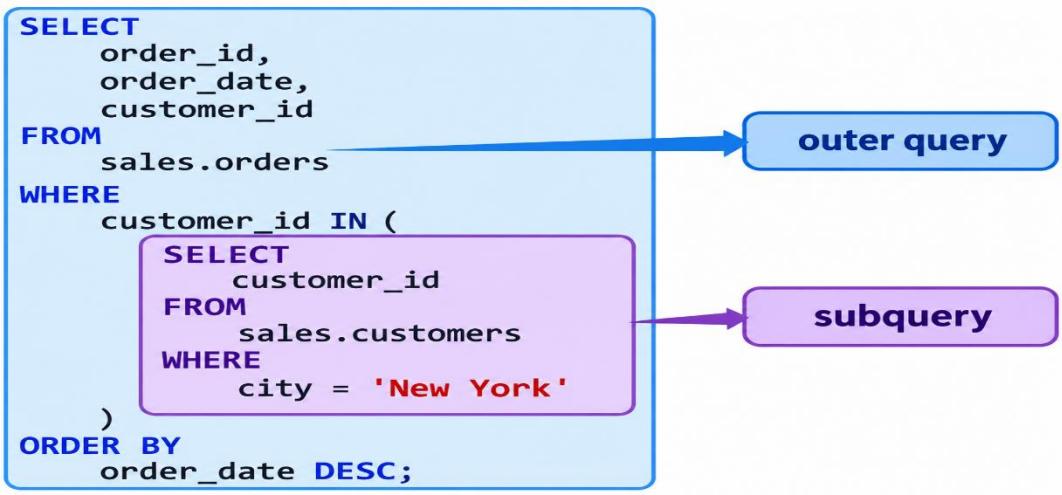
1. Subqueries and Nested Queries

A **subquery** is a query written inside another SQL query.

The inner query is executed first, and its result is used by the outer query.



Basic Structure



Types of Subqueries

Type	Description
Single-row subquery	Returns one value
Multiple-row subquery	Returns multiple values
Nested subquery	A subquery inside another subquery
Correlated subquery	Inner query depends on the outer query

Example

Find orders whose amount is greater than the average order amount.

```
SQL> SELECT
  2  Order_ID,
  3  Total_Amount,
  4  Order_Status
  5  FROM Orders
  6  WHERE Total_Amount > (
  7  SELECT AVG(Total_Amount)
  8  From Orders
  9  );

  ORDER_ID TOTAL_AMOUNT ORDER_STATUS
---------- ------------ ------------------------------
       501         1699 Pending
       503         1299 Delivered
       505         1599 Delivered
```

Explanation

The inner query:

SELECT AVG(Total_Amount) FROM Orders;

calculates the average order amount.

The outer query then displays orders whose Total_Amount is greater than that average.

2. Find Products Above Average Price

A subquery can be used to compare each product's price with the average product price.

Assuming the Product table contains:

• Product_ID

• Product_Name

• Price

Query

```
SQL> SELECT
  2  Product_ID,
  3  Product_Name,
  4  Price
  5  FROM PurplleProduct
  6  WHERE Price >
  7  (SELECT AVG(Price)
  8  from PurplleProduct);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       101
Lakme Foundation
       699

       102
Maybelline Lipstick
       749

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------

       104
Minimalist Serum
       699

       107
Fogg Perfume

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       599
```

Explanation

The inner query calculates the average price:

SELECT AVG(Price) FROM Product;

The outer query displays products whose price is greater than the calculated average.

The actual output depends on the Product data in the database.

2.1 Products Below Average Price

The same concept can be used to find products below the average price.

```
SQL> SELECT
  2  Product_ID,
  3  Product_Name,
  4  Price
  5  FROM PurplleProduct
  6  WHERE Price <
  7  (SELECT AVG(Price)
  8  from PurplleProduct);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       103
Mamaearth Face Wash
       349

       105
Loreal Shampoo
       399

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------

       106
Tresemme Conditioner
       449

       109
Dove Body Wash

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       349

       108
Nivea Body Lotion
       299

       110

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
Lakme Kajal
       299

6 rows selected.
```

3. Identify Customers with Maximum Purchases

We can use SUM() along with a subquery to identify the customer who has the highest total purchase

amount.

Step 1: Calculate Customer-wise Purchase

```
SQL> SELECT Customer_ID, SUM(Total_Amount) AS Total_Purchase FROM Orders GROUP BY Customer_ID;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
        101           1699
        102            749
        103           1299
        104            799
        105           1599

SQL>
```

Step 2: Find the Maximum Purchase

```
SQL> SELECT
  2  MAX(Total_Purchase) AS Maximum_Purchase
  3  from
  4  (SELECT
  5  Customer_ID, SUM(Total_Amount) AS Total_purchase
  6  FROM Orders
  7  GROUP BY Customer_ID);

MAXIMUM_PURCHASE
----------------
           1699
```

The inner query calculates the total purchase for each customer.

The outer query finds the maximum value among those totals.

3.1 Display Customer Details with Maximum Purchase

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.First_Name,
  4      C.Last_Name,
  5      SUM(O.Total_Amount) AS Total_Purchase
  6  FROM Customer5 C
  7  JOIN Orders O
  8      ON C.Customer_ID = O.Customer_ID
  9  GROUP BY
 10      C.Customer_ID,
 11      C.First_Name,
 12      C.Last_Name
 13  HAVING SUM(O.Total_Amount) =
 14      (SELECT MAX(Customer_Total)
 15       FROM
 16          (SELECT SUM(Total_Amount) AS Customer_Total
 17           FROM Orders
 18           GROUP BY Customer_ID));

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_PURCHASE
-------------------------------------------------- --------------
        101 Nikitha
R                                                            1699
```

Explanation

The innermost query:

SELECT SUM(Total_Amount)

FROM Orders

GROUP BY Customer_ID;

calculates the purchase amount for every customer.

The next query:

SELECT MAX(Total_Purchase)

finds the highest purchase amount.

The main query displays the customer whose purchase amount matches that maximum.

4. Complex Business Queries

Subqueries can be combined with joins and aggregate functions to solve business-related problems.

4.1 Customers Who Placed Orders Above Average Order Value

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.First_Name,
  4      C.Last_Name,
  5      O.Order_ID,
  6      O.Total_Amount
  7  FROM Customer5 C
  8  JOIN Orders O
  9      ON C.Customer_ID = O.Customer_ID
 10  WHERE O.Total_Amount >
 11      (SELECT AVG(Total_Amount)
 12       FROM Orders)
 13  ORDER BY O.Total_Amount DESC;

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          ORDER_ID TOTAL_AMOUNT
-------------------------------------------------- ---------- ------------
        101 Nikitha
R                                                         501         1699

        105 Sneha
Patel                                                     505         1599

        103 Arun
Kumar                                                     503         1299
```

Purpose

This report identifies customers who placed orders whose value is greater than the overall average order

value.

5. Find Products Whose Sales Are Above Average

The total sales of each product can be calculated from Order_Details.

```
SQL> SELECT
  2      P.Product_ID,
  3      P.Product_Name,
  4      SUM(OD.Quantity * OD.Unit_Price) AS Total_Sales
  5  FROM PurplleProduct P
  6  JOIN Order_Details OD
  7      ON P.Product_ID = OD.Product_ID
  8  GROUP BY
  9      P.Product_ID,
 10      P.Product_Name
 11  HAVING SUM(OD.Quantity * OD.Unit_Price) >
 12      (SELECT AVG(Product_Sales)
 13       FROM
 14          (SELECT
 15              SUM(Quantity * Unit_Price) AS Product_Sales
 16           FROM Order_Details
 17           GROUP BY Product_ID))
 18  ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
       101
Lakme Foundation
      2397

       105
Loreal Shampoo
      1599

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
```

Purpose

This query identifies products whose total sales are greater than the average sales generated by the

products.

6. Customer Purchase Report

A detailed customer purchase report can combine multiple tables.

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.First_Name,
  4      C.Last_Name,
  5      COUNT(O.Order_ID) AS Total_Orders,
  6      SUM(O.Total_Amount) AS Total_Purchase,
  7      AVG(O.Total_Amount) AS Average_Order_Value
  8  FROM Customer5 C
  9  JOIN Orders O
 10      ON C.Customer_ID = O.Customer_ID
 11  GROUP BY
 12      C.Customer_ID,
 13      C.First_Name,
 14      C.Last_Name
 15  ORDER BY Total_Purchase DESC;

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_ORDERS TOTAL_PURCHASE
-------------------------------------------------- ------------ --------------
AVERAGE_ORDER_VALUE
-------------------
       101 Nikitha
R                                                             1           1699
              1699

       105 Sneha
Patel                                                         1           1599
              1599

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_ORDERS TOTAL_PURCHASE
-------------------------------------------------- ------------ --------------
AVERAGE_ORDER_VALUE
-------------------

       103 Arun
Kumar                                                         1           1299
              1299

       104 Rahul
Verma                                                         1            799

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_ORDERS TOTAL_PURCHASE
-------------------------------------------------- ------------ --------------
AVERAGE_ORDER_VALUE
-------------------
              799

       102 Priya
Sharma                                                        1            749
              749
```

Report Contains

• Customer ID

• Customer Name

• Number of orders

• Total purchase amount

• Average order value

7. Advanced Product Sales Report

This report combines product and order details.

```
SQL> SELECT
  2      P.Product_ID,
  3      P.Product_Name,
  4      COUNT(DISTINCT OD.Order_ID) AS Total_Orders,
  5      SUM(OD.Quantity) AS Total_Quantity_Sold,
  6      SUM(OD.Quantity * OD.Unit_Price) AS Total_Sales,
  7      AVG(OD.Unit_Price) AS Average_Selling_Price
  8  FROM PurplleProduct P
  9  JOIN Order_Details OD
 10      ON P.Product_ID = OD.Product_ID
 11  GROUP BY
 12      P.Product_ID,
 13      P.Product_Name
 14  ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
------------ ------------------- ----------- ---------------------
       101
Lakme Foundation
           1                   3        2397                   799

       105
Loreal Shampoo
           1                   2        1599                 799.5

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
------------ ------------------- ----------- ---------------------

       103
Mamaearth Face Wash
           1                   2        1299                 649.5

       104
Minimalist Serum

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
------------ ------------------- ----------- ---------------------
           1                   1         799                   799

       102
Maybelline Lipstick
           1                   1         749                   749
```

Purpose

This report helps the business identify:

• Number of orders for each product

• Total quantity sold

• Total sales

• Average selling price

• Products generating higher sales

8. Category-wise Advanced Sales Report

Category-wise sales can be generated by combining Category, Product, and Order_Details.

```
SQL> SELECT
  2      C.Category_ID,
  3      C.Category_Name,
  4      COUNT(DISTINCT OD.Order_ID) AS Total_Orders,
  5      SUM(OD.Quantity) AS Total_Quantity_Sold,
  6      SUM(OD.Quantity * OD.Unit_Price) AS Total_Sales,
  7      AVG(OD.Unit_Price) AS Average_Selling_Price
  8  FROM Category C
  9  JOIN PurplleProduct P
 10      ON C.Category_ID = P.Category_ID
 11  JOIN Order_Details OD
 12      ON P.Product_ID = OD.Product_ID
 13  GROUP BY
 14      C.Category_ID,
 15      C.Category_Name
 16  ORDER BY Total_Sales DESC;

CATEGORY_ID
-----------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
------------ ------------------- ----------- ---------------------
           1
Makeup
           2                   4        3146                   774

           2
Skincare
           2                   3        2098                724.25

CATEGORY_ID
-----------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
------------ ------------------- ----------- ---------------------

           3
Haircare
           1                   2        1599                 799.5
```